

Data Structures

Contents

- Data
- Data Structures
- Classification of Data Structures
- Operations on Data Structures
- Selecting a Data Structure
- Algorithm
 - i. Definition
 - ii. Usage

Before we start, Let us answer these questions:

- ☐ How many cities with more than 50,000 people lie within 300 miles of Tashkent.
- ☐ How many international phone calls being made between Delhi and Tashkent every Friday evening.
- ☐ How many people in Tashkent earning more than 1000 USD per month in business.

To answer these type of questions, it is not enough to have necessary information.....

We must organize that information in a way that allows us to find the answers in time to satisfy our needs

Representing information is fundamental
to computer science.

The primary purpose of most computer programs is not to perform calculations, but to store and retrieve information — usually as fast as possible.

the study of data structures and the algorithms that manipulate them is at the heart of computer science.

Data

“Data are simply value or set of values”

e.g. name = “George”

Age=56, sex= male, passport_No.=K236598.....

Data items refers to a single unit of value. It specifies either a value of variable or constant.

A **data type** is a collection of values and set of operations that act on those values.

- set of values
- set of operations

For example,

an integer variable is a member of the integer data type.

-2,147,483,648 to 2,147,483,647 (in C language)

Addition is an example of an operation on the integer data type.

Data structure:

usually refers to an organization for data in main memory.

File structure:

an organization for data on peripheral storage, such as a disk drive or tape.

computer program design have two goals:

1. To design an algorithm that is easy to understand, code, and debug.

2. To design an algorithm that makes efficient use of the computer's resources

The choice of data structure and algorithm can make the difference between a program running in a few seconds or many days.

A solution is said to be efficient if it solves the problem within its resource constraints.

space
time

The **cost** of a solution is the amount of resources that the solution consumes.

Each data structure has costs and benefits.

A data structure requires:

- space for each data item it stores
- time to perform each basic operation
- programming effort.

Each problem has constraints on available space and time.

Only after a careful analysis of problem characteristics can we know the best data structure for the task.

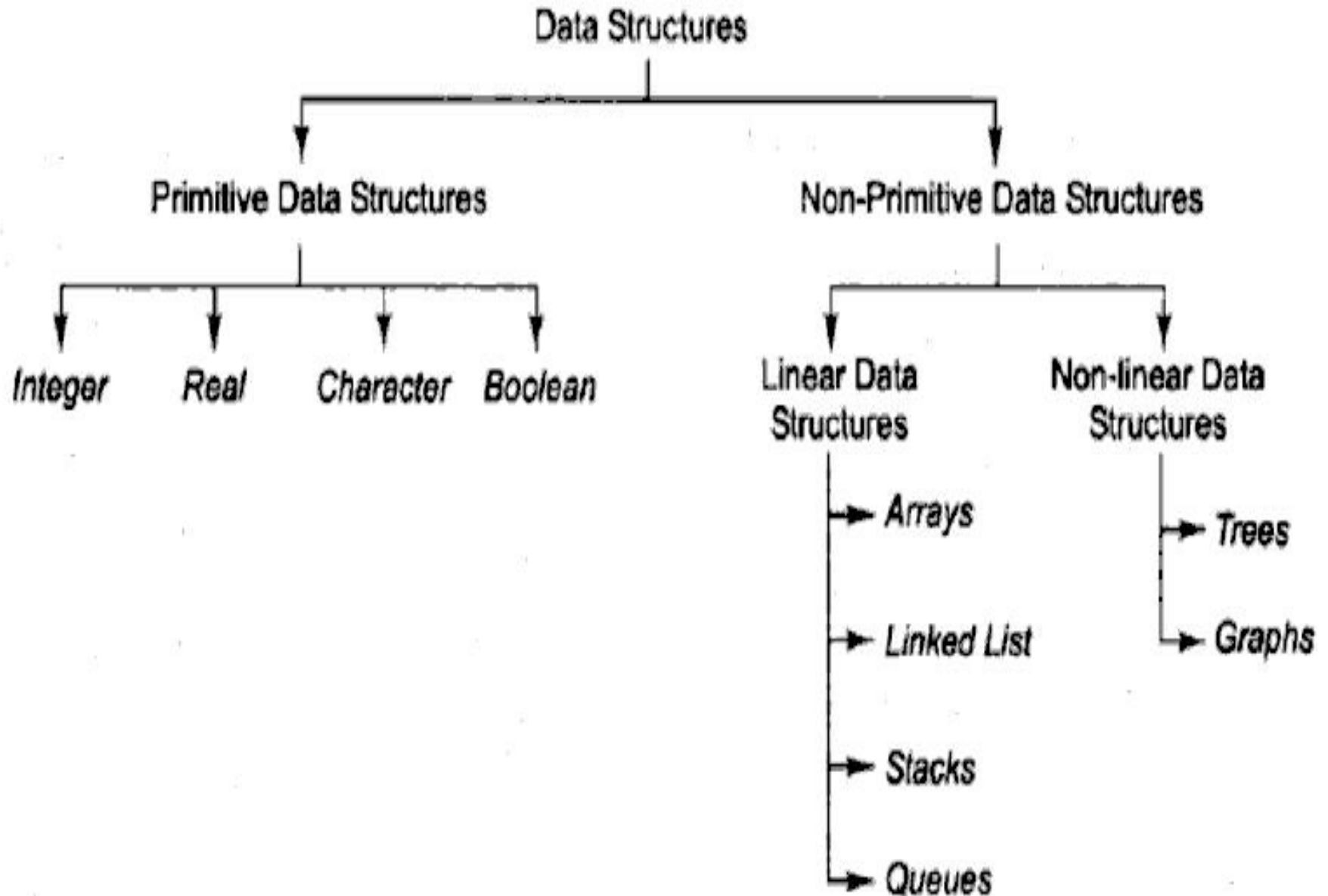
Bank example:

Start account: a few minutes

Transactions: a few seconds

Close account: overnight

Classification of Data Structures



Operations on Data Structures

1. Traversing
2. Searching
3. Inserting
4. Deleting
5. Sorting
6. Merging

Selection of Data Structure

The choice of particular data model depends on *two consideration*:

1. It must be rich enough in structure to represent the relationship between data elements

(Ex. linear relationship Vs nonlinear relationship)

1. The structure should be simple enough that one can effectively process the data when necessary

(Ex. linear relationship in Array Vs Linked List)

Following steps must be performed while selecting data structures:

1. Analysis of the problem to determine basic operations that must be supported. (ex. Finding max(heap), adding data in sorted list (linked list), etc.)
2. Quantify the resource constraints for each operation. (ex. extra space for address fields, dynamic allocation, time to access data)

Select the data structure that best meets these requirements.

As a programmer it is mandatory to choose most appropriate data structures for a program.

Problems, Algorithms, and Programs

Problems

- A **problem** is a task to be performed.
- It is best thought of in terms of inputs and matching outputs.
- The solution method should be developed only after the problem is precisely defined and thoroughly understood.
- Problems can be viewed as functions in the mathematical sense.
- A **function** is a matching between inputs (the **domain**) and outputs (the **range**).

Algorithms

An **algorithm** is a method or a process followed to solve a problem.

A problem can be solved by many different algorithms. A given algorithm solves only one problem

An algorithm possesses several properties

1. It must be correct.

must compute the desired function, converting each input to the correct output.

2. It is composed of a series of concrete steps.

Concrete means that the action described by that step is completely understood — and **achievable** — by the person or machine that must perform the algorithm.

3. There can be no ambiguity

as to which step will be performed next. Often it is the next step of the algorithm description

4. It must be composed of a finite number of steps.

If the description for the algorithm were made up of an infinite number of steps, we could never hope to write it down, nor implement it as a computer program.

5. It must terminate.

In other words, it may not go into an infinite loop.

Programs

Programs: **computer program** is a representation, of an algorithm in some programming language.

Normally the terms “algorithm” and “program” interchangeably are used, despite the fact that they are really separate concepts.

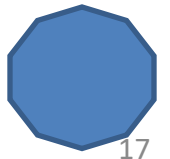
By definition, an algorithm must provide sufficient detail that it can be converted into a program when needed.

To summarize:

A **problem** is a function or a mapping of inputs to outputs.

An **algorithm** is a recipe for solving a problem whose steps are concrete and unambiguous. The algorithm must be correct, of finite length, and must terminate for all inputs.

A **program** is an instantiation of an algorithm in a computer programming language.



Abstract Data Type (ADT)

- ❑ An **ADT** describes a set of objects sharing the same properties and behaviors
 - The **properties** of an ADT are its **data**
 - The **behaviors** of an ADT are its **operations** or **functions**
- ❑ an ADT couples its data and operations
 - OOP emphasizes **data abstraction**

Encapsulation: Binding together

Abstraction : hide implementation details.

A data structure is the physical implementation of an ADT.

- ADT is a data type, where only behaviour is defined but not implementation. Examples: **Array, List, Map, Queue, Set, Stack, Table, Tree, and Vector** are ADTs
- The ADT is made of with primitive datatypes, but operation logics are hidden.
- ADT is a **logical description of how we view the data and the operations that are allowed without regard to how they will be implemented.**

What is an ADT in C++?

It is a class that has a defined set of operations and values.

Logical vs. Physical Form

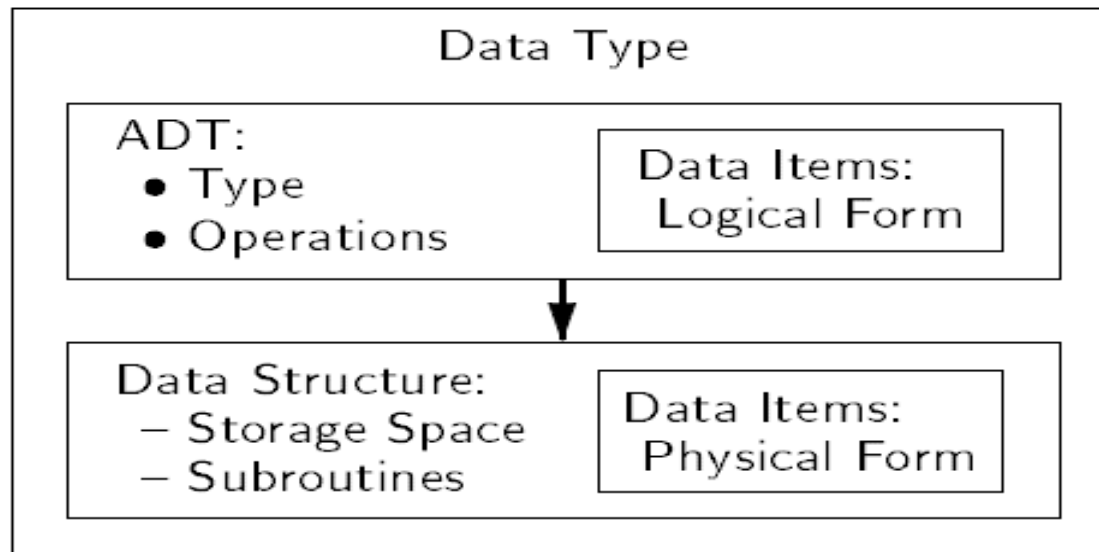
Data items have both a logical and a physical form.

Logical form:

definition of the data item within an ADT.

Physical form:

implementation of the data item within a data structure.



Asymptotic Notation

Asymptotic Complexity

(Expressed using ***Asymptotic Notation***)

- Running time of an algorithm as a function of input size n **for large n** .
- Expressed using only the **highest-order term** in the expression for the exact running time.
 - Instead of exact running time, say $\Theta(n^2)$.
- Describes behavior of function in the limit.

Asymptotic Notation

- Θ , O , Ω ,
- Defined for functions over the natural numbers.
 - Ex: $f(n) = \Theta(n^2)$.
 - Describes how $f(n)$ grows in comparison to n^2 .

Define a **set** of functions;

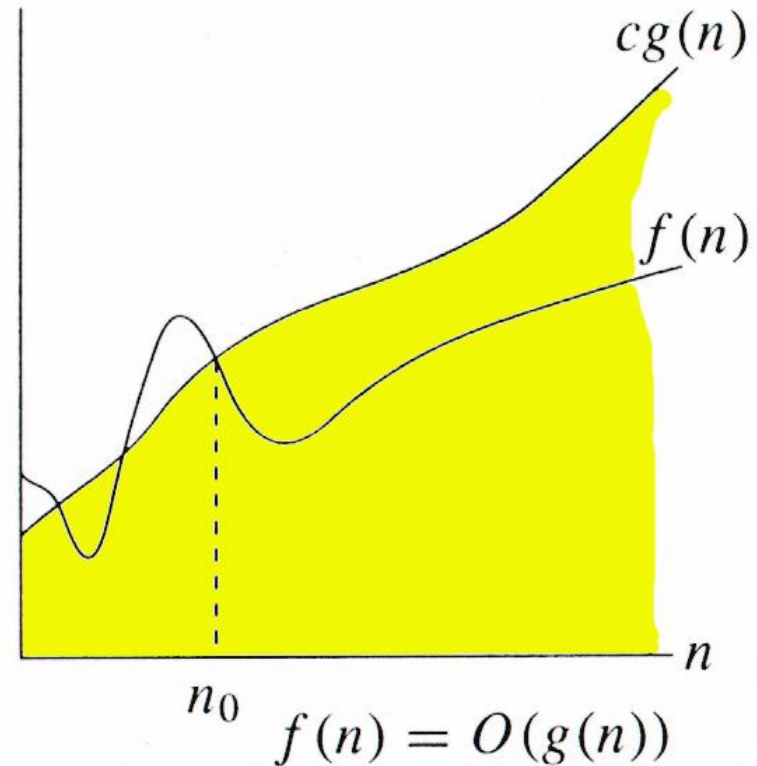
- in practice used to compare two function sizes.
- The notations describe different rate-of-growth relations between the defining function and the defined set of functions.

O-notation

For function $g(n)$, we define $O(g(n))$, big-O of n , as the set:

$$O(g(n)) = \{f(n) : \\ \exists \text{ positive constants } c \text{ and } n_0, \text{ such} \\ \text{that } \forall n \geq n_0, \\ \text{we have } 0 \leq f(n) \leq cg(n) \}$$

Intuitively: Set of all functions whose *rate of growth* is the same as or lower than that of $g(n)$.



$g(n)$ is an *asymptotic upper bound* for $f(n)$.

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = O(g(n)).$$

$$\Theta(g(n)) \subset O(g(n)).$$

Examples

$O(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that } \forall n \geq n_0, \text{ we have } 0 \leq f(n) \leq cg(n) \}$

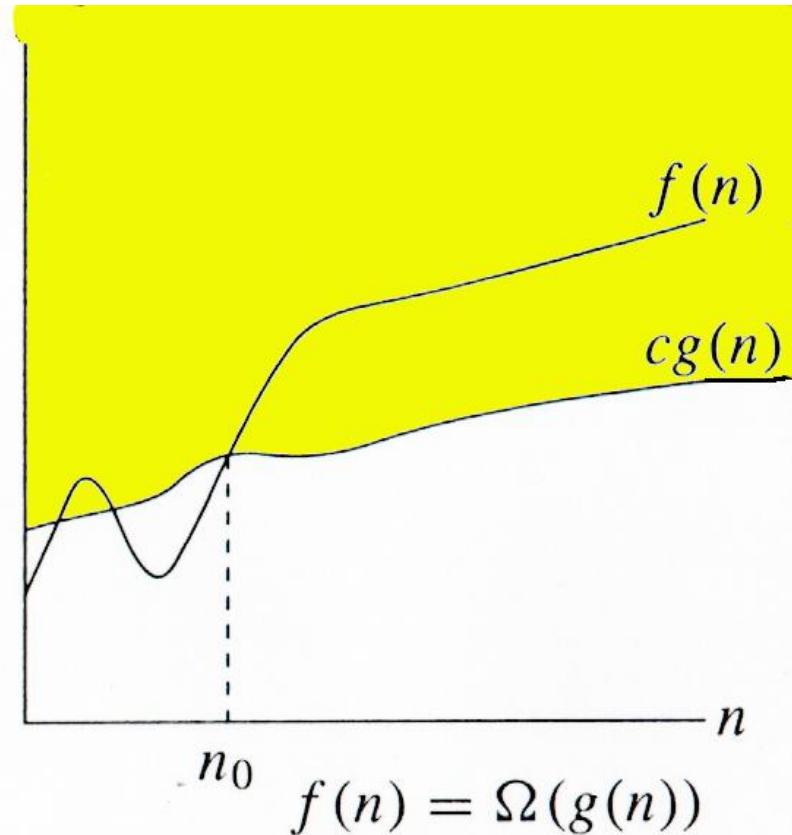
- Any linear *function* $an + b$ is in $O(n^2)$. How?
- Show that $3n^3 = O(n^4)$ for appropriate c and n_0 .

Ω -notation

For function $g(n)$, we define $\Omega(g(n))$, big-Omega of n , as the set:

$\Omega(g(n)) = \{f(n) :$
 $\exists$ positive constants c and n_0 , such
that $\forall n \geq n_0$,
we have $0 \leq cg(n) \leq f(n)\}$

Intuitively: Set of all functions whose *rate of growth* is the same as or higher than that of $g(n)$.



$g(n)$ is an *asymptotic lower bound* for $f(n)$.

$$f(n) = \Theta(g(n)) \Rightarrow f(n) = \Omega(g(n)).$$

$$\Theta(g(n)) \subset \Omega(g(n)).$$

Example

$\Omega(g(n)) = \{f(n) : \exists \text{ positive constants } c \text{ and } n_0, \text{ such that}$
 $\forall n \geq n_0, \text{ we have } 0 \leq cg(n) \leq f(n)\}$

- $n^3 + 2n = \Omega(n)$. Choose c and n_0 .

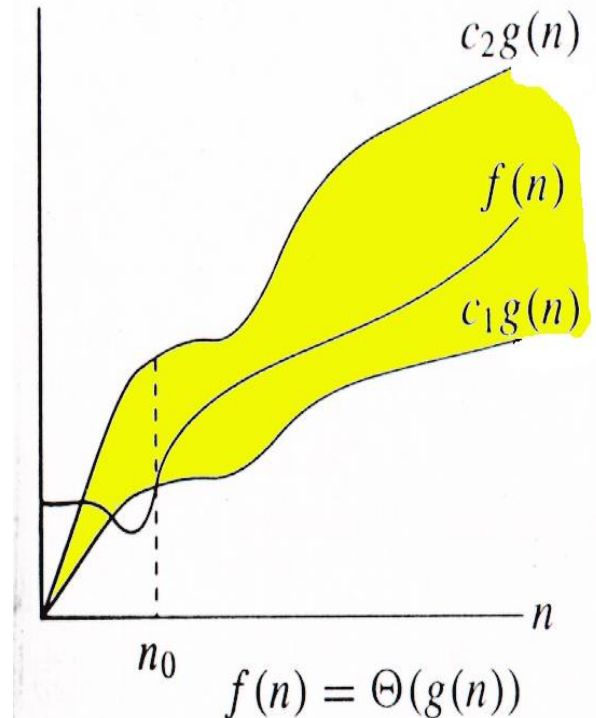
Θ -notation

For function $g(n)$, we define $\Theta(g(n))$, big-Theta of n , as the set:

$$\Theta(g(n)) = \{f(n) : \exists \text{ positive constants } c_1, c_2, \text{ and } n_0, \text{ such that } \forall n \geq n_0, \text{ we have } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n)\}$$

Intuitively: Set of all functions that have the same rate of growth as $g(n)$.

$g(n)$ is an **asymptotically tight bound** for $f(n)$.



Example

$\Theta(g(n)) = \{f(n) : \exists \text{ positive constants } c_1, c_2, \text{ and } n_0, \text{ such that } \forall n \geq n_0, \quad 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$

- $10n^2 - 3n = \Theta(n^2)$
- What constants for n_0 , c_1 , and c_2 will work?
- Make c_1 a little smaller than the leading coefficient, and c_2 a little bigger.
- *To compare orders of growth, look at the leading term.*

Video Lecture of the topic can be found at

Introduction to DATA STRUCTURES

<https://youtu.be/AtMUVr3X6H8>

Data Structures - Abstract Data Types

<https://youtu.be/y58iGerTlt8>